LAPORAN TUGAS KECIL
STRATEGI ALGORITMA

VOXELIZATION OBJEK 3D MENGGUNAKAN OCTREE

Dipersiapkan oleh:
13524025 - Moh. Hafizh Irham Perdana
13524077 - Yavie Azka Putra Araly

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi **2**
Bab 1 Pendahuluan **3**
A. Dasar Teori 3
B. Deskripsi Program 4
Bab 2 Implementasi Program **5**
A. Struktur Data 5
B. Implementasi 5
Bab 3 Hasil dan Pengujian **11**
A. Pengujian ke-1 11
B. Pengujian ke-2 12
C. Pengujian ke-3 12
D. Pengujian ke-4 13
E. Pengujian ke-5 14
Referensi **17**
Lampiran **17**

Bab 1
Pendahuluan

A. Dasar Teori

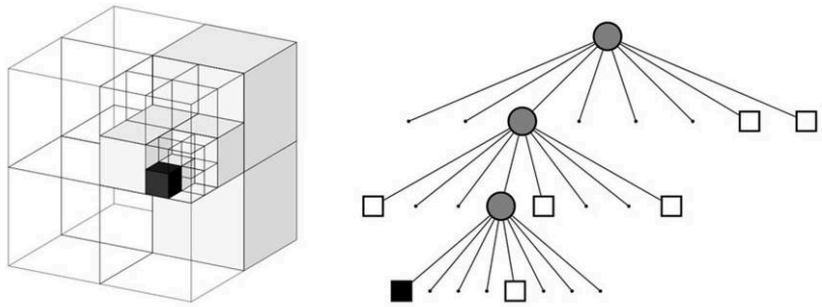
1. Algoritma Divide and Conquer

   Algoritma Divide and Conquer adalah teknik desain algoritma yang memecah masalah besar menjadi sub-masalah yang lebih kecil. Algoritma ini terdiri dari tiga tahapan: (1) Divide: artinya membagi persoalan menjadi beberapa sub-persoalan yang memilki kemiripan dengan persoalan semula namun dengan ukuran yang lebih kecil, (2) Conquer: artinya menyelesaikan masing-masing sub-persoalan secara langsung apabila sudah cukup kecil atau secara rekursif apabila masih terlalu besar, (3) Combine: artinya menggabungkan masing-masing solusi sub-persoalan sehingga membentuk solusi persoalan awal.

   Algoritma ini cukup efektif dan dapat diaplikasikan dalam berbagai persoalan karena setiap sub persoalan akan diselesaikan secara independen, terutama apabila menerapkan konkurensi pada setiap sub-persoalan. Dengan ini, kita dapat memaksimalkan fungsi setiap sumber daya komputasi dan meminimalisasi waktu komputasi. Secara kompleksitas, algoritma ini jauh lebih baik dibanding algoritma *brute force* walaupun akan berbeda untuk setiap persoalan.

2. Octree

   Octree merupakan struktur data hierarkis yang secara rekursif membagi objek tiga dimensi menjadi delapan bagian atau yang disebut *octant*. Setiap node di dalam struktur data ini dapat berisi sebuah data atau terbagi lagi menjadi node anak. Singkatnya, Octree merupakan struktur data *tree* yang dimodifikasi untuk merepresentasikan sebuah objek 3D ke dalam sub-kubus pembentuknya.

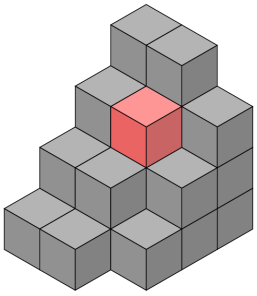


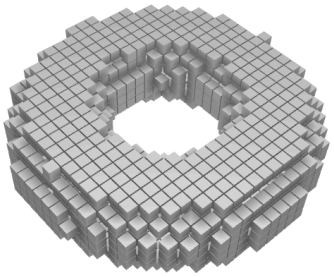
   Struktur data ini sering dipakai dalam grafika komputer, simulasi fisika, data geospasial, dan *robotic pathfinding*. Octree merupakan *essential tool* untuk

mengoptimasi efisiensi ketika berhubungan dengan objek 3D. Struktur pohonnya mempermudah kita untuk melakukan modifikasi spasial dengan presisi dan mengurangi konsumsi memori.

3. Voxel

Voxel atau yang sering digambarkan sebagai piksel volumetrik merupakan objek berbasis poligon 3D yang berada dalam *grid* kubus dengan konsep yang sama seperti piksel 2D dalam *grid* persegi. Voxel menempati ruang tiga dimensi yang didasarkan pada koordinat *grid* dimana semua voxel memiliki ukuran yang sama.





B. Deskripsi Program

Program merupakan aplikasi yang dapat mengubah objek 3D *polygon mesh* menjadi objek voxel dalam file .obj. Program menggunakan algoritma *divide and conquer* dengan mengonversi file .obj menjadi struktur data Octree. Kemudian program akan membentuk sebuah Octree kubus yang membungkus objek 3D tersebut. Node akan dibentuk apabila terdapat sub-kubus yang beririsan dengan objek secara rekursif dengan kedalaman yang telah ditentukan sebelumnya. Irisan objek ditentukan dengan membentuk segitiga berdasarkan koordinat (x, y, z), vertice v, dan faces f. Apabila tidak terdapat bagian objek yang beririsan maka sub-kubus akan dihilangkan. Program mengaplikasikan konkurensi pada setiap node dan dibuat dalam bahasa C++.

Bab 2
Implementasi Program

A. Struktur Data

1. **`Point`** (Struct):

   Merupakan struktur data fundamental untuk menyimpan koordinat spasial tiga dimensi. Struct ini memiliki tiga atribut bertipe float, yaitu x, y, dan z. Point digunakan untuk merepresentasikan koordinat vertex dari model .obj, titik pusat (center) dari sebuah node octree, serta untuk perhitungan vektor dalam deteksi tabrakan.

2. **`OctreeNode`** (Class):

   Merepresentasikan satu simpul (ruang kubus) di dalam pohon Octree. Class ini menyimpan informasi batas ruang dan anak-anaknya:

   - **`center`** (Point): Titik tengah dari kubus.
   - **`size`** (float): Panjang sisi kubus.
   - **`children`** (OctreeNode*[8]): Array of pointers yang menyimpan referensi ke 8 sub-oktan (anak) dari node tersebut.
   - **`faceIndices`** (vector<int>): Menyimpan indeks dari faces (segitiga) dari model 3D asli yang memotong/bersinggungan dengan ruang kubus node ini.
   - **`isLeaf`** & **`isVoxel`** (bool): Penanda status node. isVoxel menandakan bahwa node ini adalah kubus terkecil pada kedalaman maksimum yang akan dirender sebagai output voxel.

3. **`Octree`** (Class):

   Struktur data utama yang mengelola keseluruhan pohon dan data objek:

   - **`root`** (OctreeNode*): Simpul akar yang melingkupi seluruh bounding box dari objek 3D.
   - **`vertex`** & **`face`** (vector): Menyimpan keseluruhan data verteks dan muka segitiga yang diparsing langsung dari file .obj masukan.
   - **`maxDepth`** (int): Batas kedalaman maksimum pohon sesuai input pengguna.
   - Struktur pencatatan statistik (seperti **`nodeCountPerDepth`** dan **`prunedNodes`**) untuk keperluan pelaporan di CLI.

B. Implementasi

Program ini menggunakan algoritma Divide and Conquer untuk memecah ruang 3D (kotak pembatas utama/ bounding box) menjadi bagian-bagian yang lebih

kecil secara rekursif hingga mencapai kedalaman maksimum yang ditentukan pengguna. Transformasi ini berfokus pada pemotongan ruang yang hanya bersinggungan dengan permukaan objek. Langkah-langkah penyelesaiannya adalah sebagai berikut:

1. **Inisialisasi**

   Sebelum rekursi dimulai, program mencari nilai minimum dan maksimum dari koordinat x, y, dan z dari seluruh verteks masukan untuk membentuk bounding box utama berupa kubus (node root). Seluruh indeks faces dari objek .obj dimasukkan ke dalam node root ini sebagai titik awal pengecekan.

```
// Fungsi Inisialisasi: Membuat Bounding Box Utama
FUNCTION BuildOctree(vertices, faces, maxDepth)
    // 1. Cari nilai minimum dan maksimum dari seluruh verteks
    minX, minY, minZ = INFINITY
    maxX, maxY, maxZ = -INFINITY

    FOR EACH v IN vertices:
        minX = MIN(minX, v.x); maxX = MAX(maxX, v.x)
        minY = MIN(minY, v.y); maxY = MAX(maxY, v.y)
        minZ = MIN(minZ, v.z); maxZ = MAX(maxZ, v.z)
    END FOR

    // 2. Hitung titik pusat dan ukuran kubus terbesar (Root)
    center = Point((minX + maxX)/2, (minY + maxY)/2, (minZ +
maxZ)/2)
    size = MAX(maxX - minX, maxY - minY, maxZ - minZ)

    // 3. Buat simpul akar (Root Node)
    root = NEW OctreeNode(center, size)
    root.faceIndices = faces // Masukkan semua indeks segitiga
ke root

    // 4. Panggil fungsi rekursif Divide and Conquer
    Voxelize(root, maxDepth)
END FUNCTION
```

2. **Langkah Divide (Pembagian Ruang)**

   Pada fungsi rekursif voxelize(), sebuah ruang kubus yang direpresentasikan oleh **`OctreeNode`** dibagi secara simetris menjadi 8 kubus yang lebih kecil (sub-oktan). Panjang sisi (size) setiap anak dikurangi menjadi setengah dari sisi induknya (**`node->getSize() / 2`**). Titik pusat (center) setiap anak dihitung dengan menggeser titik pusat induk ke 8 arah kuadran berbeda sejauh seperempat ukuran induk (offsets[8]).

```
// Fungsi Divide and Conquer Rekursif
FUNCTION Voxelize(node, depth)
    // BASE CASE: Jika kedalaman sudah mencapai 0
    IF depth == 0 THEN
        node.isLeaf = TRUE
        node.isVoxel = TRUE
        RETURN
    END IF

    // DIVIDE: Tentukan titik pusat untuk 8 sub-oktan (anak
kubus)
    offsets = Array of 8 arah kuadran ruang 3D (x, y, z)
    anyChildCreated = FALSE

    FOR i = 0 TO 7:
        // Hitung parameter anak
        childSize = node.size / 2
        childCenter = node.center + offsets[i] * (node.size / 4)

        childNode = NEW OctreeNode(childCenter, childSize)
        intersects = FALSE

        // CONQUER: Cek persinggungan antara sub-oktan dan
segitiga
        FOR EACH faceIdx IN node.faceIndices:
            segitiga = AmbilSegitiga(faceIdx)

            // Menggunakan Separating Axis Theorem (SAT)
            IF TriangleIntersectsCube(segitiga, childCenter,
childSize/2) THEN
                childNode.faceIndices.APPEND(faceIdx)
                intersects = TRUE
            END IF
        END FOR

        // COMBINE & PRUNING
        IF intersects == TRUE THEN
            // Jika memotong permukaan, simpan anak dan
lanjutkan rekursi
            node.children[i] = childNode
            Voxelize(childNode, depth - 1)
            anyChildCreated = TRUE
        ELSE
            // PRUNING: Jika ruang kosong (tidak memotong
objek), hapus anak
            DELETE childNode
            node.children[i] = NULL
        END IF
    END FOR

    // Tandai status node induk
    IF anyChildCreated == TRUE THEN
        node.isLeaf = FALSE
```

```
    END IF
END FUNCTION
```

3. **Langkah Conquer (Deteksi Interseksi)**

Setelah 8 sub-oktan terbentuk, program harus memfilter oktan mana yang benar-benar memuat permukaan dari model 3D.

- Untuk setiap anak (sub-oktan), program mengecek apakah ada faces (segitiga) dari node induk yang bersinggungan (intersect) dengan ruang sub-oktan tersebut.
- Pengecekan persinggungan ini menggunakan fungsi **`triangleIntersects()`** yang mengimplementasikan Separating Axis Theorem (SAT). Algoritma menguji tumpang tindih (overlap) proyeksi segitiga dan kubus pada 13 sumbu berbeda (3 sumbu utama kubus, 1 sumbu normal segitiga, dan 9 sumbu cross product dari edge keduanya).
- Jika bersinggungan, indeks face tersebut diwariskan ke **`faceIndices`** milik anak.

```
// Fungsi pembantu untuk mengecek tumpang tindih (overlap) pada
satu sumbu
FUNCTION OverlapOnAxis(v0, v1, v2, halfSize, axis)
    // 1. Proyeksikan titik-titik segitiga ke sumbu menggunakan
Dot Product
    p0 = DotProduct(v0, axis)
    p1 = DotProduct(v1, axis)
    p2 = DotProduct(v2, axis)

    // 2. Cari jarak minimum dan maksimum dari proyeksi segitiga
    triMin = MIN(p0, p1, p2)
    triMax = MAX(p0, p1, p2)

    // 3. Hitung jari-jari proyeksi kubus pada sumbu tersebut
    r = (halfSize.x * ABS(axis.x)) + (halfSize.y * ABS(axis.y))
+ (halfSize.z * ABS(axis.z))

    // 4. Periksa apakah proyeksi segitiga berada di luar
rentang kubus (-r hingga r)
    IF triMin > r OR triMax < -r THEN
        RETURN FALSE // Ditemukan sumbu pemisah (Separating
Axis), tidak ada tabrakan
    END IF

    RETURN TRUE // Terjadi tumpang tindih pada sumbu ini
END FUNCTION
```

```
// Fungsi utama pengecekan persinggungan segitiga dan kubus
(Sub-oktan)
FUNCTION TriangleIntersectsCube(segitiga, center, halfSize)
    // 1. Pindahkan koordinat verteks segitiga ke ruang lokal
kubus (titik acuan 0,0,0)
    v0 = segitiga.v0 - center
    v1 = segitiga.v1 - center
    v2 = segitiga.v2 - center

    // 2. Tentukan vektor sisi (edges) dari segitiga
    e0 = v1 - v0
    e1 = v2 - v1
    e2 = v0 - v2

    // Definisikan 3 sumbu utama kubus (X, Y, Z)
    axisX = Point(1, 0, 0)
    axisY = Point(0, 1, 0)
    axisZ = Point(0, 0, 1)

    // --- MULAI PENGUJIAN 13 SUMBU (SAT) ---

    // Tes 1: Uji 9 sumbu hasil Cross Product antara sisi
segitiga (3) dan sumbu kubus (3)
    axes9 = [
        CrossProduct(e0, axisX), CrossProduct(e0, axisY),
CrossProduct(e0, axisZ),
        CrossProduct(e1, axisX), CrossProduct(e1, axisY),
CrossProduct(e1, axisZ),
        CrossProduct(e2, axisX), CrossProduct(e2, axisY),
CrossProduct(e2, axisZ)
    ]
    FOR EACH ax IN axes9:
        IF OverlapOnAxis(v0, v1, v2, halfSize, ax) == FALSE THEN
            RETURN FALSE
        END IF
    END FOR

    // Tes 2: Uji 3 sumbu utama kubus (Normal dari sisi-sisi
kubus)
    IF OverlapOnAxis(v0, v1, v2, halfSize, axisX) == FALSE THEN
RETURN FALSE
    IF OverlapOnAxis(v0, v1, v2, halfSize, axisY) == FALSE THEN
RETURN FALSE
    IF OverlapOnAxis(v0, v1, v2, halfSize, axisZ) == FALSE THEN
RETURN FALSE

    // Tes 3: Uji sumbu normal dari bidang segitiga itu sendiri
    segitigaNormal = CrossProduct(e0, e1)
    IF OverlapOnAxis(v0, v1, v2, halfSize, segitigaNormal) ==
FALSE THEN RETURN FALSE

    // Jika lolos pengujian di semua 13 sumbu tanpa
mengembalikan FALSE,
```

```
    // maka dipastikan terjadi persinggungan (intersect)
    RETURN TRUE
END FUNCTION
```

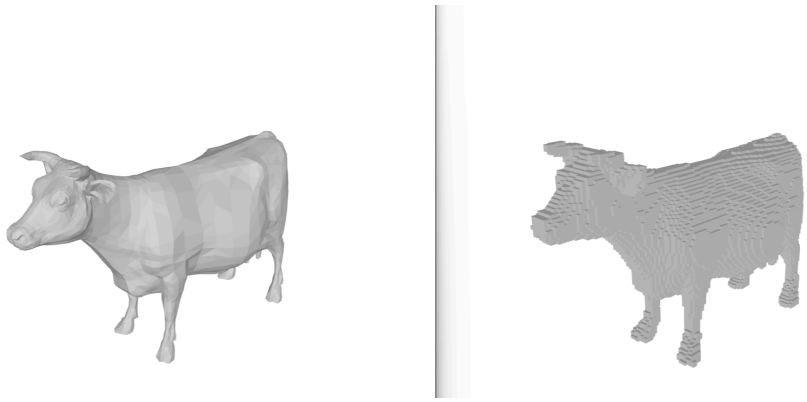
4. **Langkah Combine & Base Case (Penggabungan dan Penghentian)**
   - Basis 1 (Voxel Terbentuk): Jika rekursi telah mencapai sisa kedalaman nol (depth = 0), maka proses pemecahan dihentikan. Node anak tersebut ditandai sebagai leaf dan voxel (**`isVoxel = true`**), yang nantinya akan dicetak sebagai kubus kecil pada file output.
   - Basis 2 (Pruning / Pemangkasan): Jika pada langkah Conquer sebuah sub-oktan terbukti tidak bersinggungan dengan segitiga apa pun, maka node anak tersebut akan dihapus dari memori (delete) dan diabaikan (di-prune). Cabang ini tidak akan ditelusuri lebih lanjut, sehingga sangat menghemat komputasi dan memori ruang kosong.
   - Combine: Pohon akan terbentuk dengan sendirinya melalui pointer children. Rekursi untuk mengeksplorasi cabang-cabang anak dapat dilakukan secara paralel menggunakan instruksi OpenMP (**`#pragma omp parallel for`**) pada level kedalaman tertentu untuk mempercepat waktu eksekusi (Spesifikasi Bonus).

Bab 3
Hasil dan Pengujian

Dalam pengujian, kedalaman Octree maksimum yang digunakan adalah 7. Optimasi paralel digunakan agar waktu komputasi lebih cepat. Semua objek dibawah dihasilkan dari command berikut:

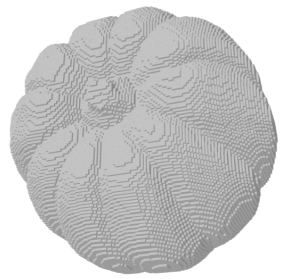
```
bin/voxelizer test/<file_input>.obj 7 test/<file_output>.obj 1
```

A. Pengujian ke-1



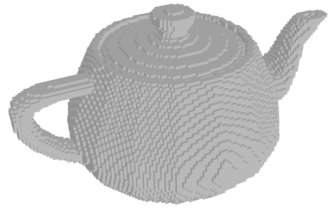
```
Input diterima dengan baik!
File OBJ target : test/cow.obj
Kedalaman Octree: 7
-----------------------------------
Output disimpan di: test/cowoutput.obj
Waktu eksekusi: 392 ms
STATISTIK:
Banyaknya voxel yang terbentuk : 22864
Banyaknya vertex yang terbentuk: 182912
Banyaknya faces yang terbentuk : 274368
Node octree yang terbentuk:
1 : 8
2 : 20
3 : 88
4 : 332
5 : 1362
6 : 5627
7 : 22864
Node yang tidak perlu ditelusuri:
1 : 0
2 : 44
3 : 72
4 : 372
5 : 1294
6 : 5269
7 : 22152
Kedalaman octree: 7
Waktu perhitungan: 392 ms
Path hasil .obj disimpan: test/cowoutput.obj
```

B. Pengujian ke-2



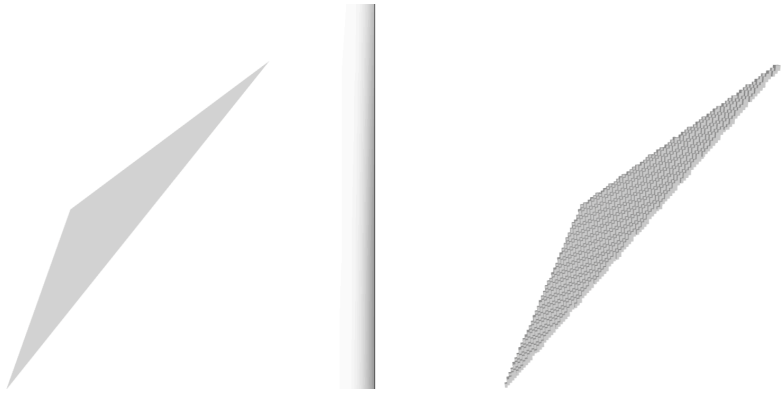
```
Input diterima dengan baik!
File OBJ target : test/pumpkin.obj
Kedalaman Octree: 7
-----------------------------------
Output disimpan di: test/pumpkinoutput.obj
Waktu eksekusi: 1258 ms
STATISTIK:
Banyaknya voxel yang terbentuk : 74750
Banyaknya vertex yang terbentuk: 598000
Banyaknya faces yang terbentuk : 897000
Node octree yang terbentuk:
1 : 8
2 : 56
3 : 277
4 : 1123
5 : 4626
6 : 18637
7 : 74750
Node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 171
4 : 1093
5 : 4358
6 : 18371
7 : 74346
Kedalaman octree: 7
Waktu perhitungan: 1258 ms
Path hasil .obj disimpan: test/pumpkinoutput.obj
```

C. Pengujian ke-3



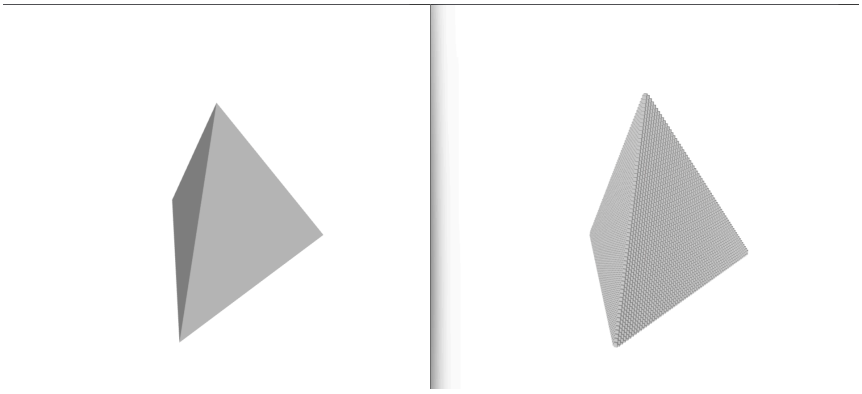
```
Input diterima dengan baik!
File OBJ target : test/teapot.obj
Kedalaman Octree: 7
-----------------------------------
Output disimpan di: test/teapotoutput.obj
Waktu eksekusi: 489 ms
STATISTIK:
Banyaknya voxel yang terbentuk : 29565
Banyaknya vertex yang terbentuk: 236520
Banyaknya faces yang terbentuk : 354780
Node octree yang terbentuk:
1 : 8
2 : 24
3 : 108
4 : 418
5 : 1748
6 : 7303
7 : 29565
Node yang tidak perlu ditelusuri:
1 : 0
2 : 40
3 : 84
4 : 446
5 : 1596
6 : 6681
7 : 28859
Kedalaman octree: 7
Waktu perhitungan: 489 ms
Path hasil .obj disimpan: test/teapotoutput.obj
```

D. Pengujian ke-4



```
Input diterima dengan baik!
File OBJ target : test/line.obj
Kedalaman Octree: 7
-----------------------------------
Output disimpan di: test/lineoutput.obj
Waktu eksekusi: 77 ms
STATISTIK:
Banyaknya voxel yang terbentuk : 3824
Banyaknya vertex yang terbentuk: 30592
Banyaknya faces yang terbentuk : 45888
Node octree yang terbentuk:
1 : 7
2 : 14
3 : 32
4 : 92
5 : 287
6 : 1018
7 : 3824
Node yang tidak perlu ditelusuri:
1 : 1
2 : 42
3 : 80
4 : 164
5 : 449
6 : 1278
7 : 4320
Kedalaman octree: 7
Waktu perhitungan: 77 ms
Path hasil .obj disimpan: test/lineoutput.obj
```

E. Pengujian ke-5



```
Input diterima dengan baik!
File OBJ target : test/tetrahedron.obj
Kedalaman Octree: 7
-----------------------------------
Output disimpan di: test/tetrahedronoutput.obj
Waktu eksekusi: 913 ms
STATISTIK:
Banyaknya voxel yang terbentuk : 56834
Banyaknya vertex yang terbentuk: 454672
Banyaknya faces yang terbentuk : 682008
Node octree yang terbentuk:
1 : 8
2 : 42
3 : 194
4 : 834
5 : 3458
6 : 14082
7 : 56834
Node yang tidak perlu ditelusuri:
1 : 0
2 : 22
3 : 142
4 : 718
5 : 3214
6 : 13582
7 : 55822
Kedalaman octree: 7
Waktu perhitungan: 913 ms
Path hasil .obj disimpan: test/tetrahedronoutput.obj
```

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil Konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj dan mengembalikan output	✓	
6	[Bonus] Program menggunakan concurrency	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Moh. Hafizh Irham Perdana

/

Yavie Azka Putra Araly

Referensi

- https://dyn4j.org/2010/01/sat/
- https://programmerart.weebly.com/separating-axis-theorem.html
- https://www.geeksforgeeks.org/dsa/introduction-to-divide-and-conquer-algorithm/
- https://www.linkedin.com/pulse/octrees-standard-spatial-data-structure-its-potential-pelletier-jelse/

Lampiran

- https://github.com/YavieAzka/Tucil2_113524025_113524077